

Engineering-First Web Development Curriculum

From MERN Tutorial-Follower → Staff-Track Software Engineer

Design principle: Every module = Problem → Internals → Code → Production → Practice. No topic appears until its prerequisite mental model exists. Technology is the vehicle; engineering judgment is the payload.

PHASE 0 — Computing Foundations (The Substrate)

Most bootcamp grads skip this and it caps their ceiling permanently. This phase exists so nothing later is "magic."

Module 0.1 — How Computers Actually Run Your Code

- **Submodules:** Memory (stack vs heap), CPU execution model, processes vs threads, compilation vs interpretation vs JIT, how V8 executes JS
- **Learning Objectives:** Explain why JS is single-threaded but non-blocking; explain garbage collection basics
- **Why this exists:** You cannot reason about Node.js concurrency, memory leaks, or performance without this
- **Industry relevance:** Debugging memory leaks in production Node services requires this model
- **Misconceptions cleared:** "JavaScript is single-threaded so it can't do async" (event loop vs call stack confusion)
- **Practical Exercise:** Trace a memory leak in a given Node snippet; explain call stack overflow vs heap exhaustion
- **Difficulty:** Medium
- **Prerequisites:** None

Module 0.2 — Networking Fundamentals

- **Submodules:** OSI model (practical subset), TCP vs UDP, DNS resolution, TLS handshake, ports/sockets, how a URL becomes a response
- **Learning Objectives:** Trace what happens between typing a URL and pixels appearing on screen

- **Why this exists:** HTTP, REST, WebSockets, CORS, and deployment all sit on top of this
- **Industry relevance:** Debugging "works locally, fails in prod" networking issues, CORS errors, SSL cert failures
- **Practical Exercise:** Use `curl -v`, `dig`, and browser DevTools Network tab to manually trace a full request lifecycle
- **Difficulty:** Medium
- **Prerequisites:** None

Module 0.3 — Operating Systems & Shell Literacy

- **Submodules:** Filesystem model, processes, environment variables, permissions, piping, background jobs
- **Learning Objectives:** Comfortable in a Linux shell without a GUI; understand `.env`, `PATH`, process signals
- **Why this exists:** Every deployment, Docker container, and CI pipeline is Linux underneath
- **Practical Exercise:** Debug a "command not found" and a permission-denied error using only shell tools
- **Difficulty:** Easy–Medium

Skills after Phase 0: You stop treating your runtime as a black box. Debugging shifts from "restart and pray" to "form a hypothesis about what layer failed."

PHASE 1 — Web Fundamentals (Before Any Framework)

Module 1.1 — HTTP: The Actual Protocol

- **Submodules:** Request/response anatomy, methods & idempotency, status codes (real semantics, not memorized lists), headers (Content-Type, Cache-Control, ETag), statelessness, HTTP/1.1 vs HTTP/2 vs HTTP/3
- **Why this exists:** Every REST API, every browser behavior, every caching bug traces back here
- **Industry relevance:** Caching bugs, wrong status codes in APIs, CORS misunderstandings are the #1 junior-dev bug source
- **Misconceptions cleared:** "POST is for creating, PUT is for updating" (idempotency is the real distinction); "200 means success" (200 with error body is a common anti-pattern)
- **Practical Exercise:** Given a buggy API, identify which HTTP semantics are violated (wrong idempotency, wrong caching headers)
- **Difficulty:** Medium
- **Prerequisites:** 0.2

Module 1.2 — Browser Internals

- **Submodules:** Critical rendering path (HTML parse → DOM → CSSOM → Render Tree → Layout → Paint → Composite), reflow vs repaint, event loop in the browser (microtasks vs macrotasks vs rendering steps), how `<script defer/async>` changes parsing
- **Why this exists:** This is the foundation for understanding why React's Virtual DOM exists at all
- **Industry relevance:** Explains jank, layout thrashing, and why certain CSS/JS patterns tank performance
- **Practical Exercise:** Profile a janky page in Chrome DevTools Performance tab; identify a forced synchronous layout
- **Difficulty:** Medium-Hard
- **Prerequisites:** 1.1

Module 1.3 — JavaScript, Deeply

- **Submodules:**
 - Execution context, scope chain, closures (mechanism, not just definition)
 - `this` binding rules (all 4, with edge cases)
 - Prototypal inheritance vs classical (why `class` is sugar)
 - Event loop: call stack, Web APIs, callback queue, microtask queue — with `setTimeout` vs `Promise` ordering traced by hand
 - Coercion rules and why `==` is dangerous (not "just use `===`" — the actual algorithm)
 - Higher-order functions, currying, composition
 - Memory: closures causing leaks, WeakMap use cases
- **Why this exists:** Every framework bug ("why did my `useEffect` fire twice," "why is this undefined") is a JS fundamentals gap in disguise
- **Misconceptions cleared:** "async/await replaces promises" (it's sugar over the same microtask mechanism); "arrow functions are just shorter syntax" (lexical `this` matters)
- **Practical Exercise:** Predict `console.log` output order for a mixed sync/promise/setTimeout snippet (classic interview question); implement `debounce`, `throttle`, and a basic `Promise` from scratch
- **Difficulty:** Hard
- **Prerequisites:** 0.1, 1.2

Module 1.4 — TypeScript as an Engineering Tool (Not "JS with Types")

- **Submodules:** Structural typing vs nominal typing, type inference, generics (why and when), utility types, discriminated unions, `unknown` vs `any`, declaration merging, module resolution

- **Why this exists:** TS is how large codebases stay maintainable; understanding *why* type systems exist (catching entire classes of runtime errors at compile time) matters more than syntax memorization
- **Industry relevance:** Nearly all serious MERN codebases in 2026 are TypeScript-first
- **Practical Exercise:** Refactor a loosely-typed API layer into discriminated unions that make invalid states unrepresentable
- **Difficulty:** Medium-Hard
- **Prerequisites:** 1.3

Skills after Phase 1: You can explain *why* a bug happens at the language/browser level, not just patch symptoms.

PHASE 2 — Server-Side Engineering (Node.js & Express)

Module 2.1 — Node.js Internals

- **Submodules:** libuv and the event loop phases (timers, poll, check), why Node is single-threaded but I/O is non-blocking, `process.nextTick` vs microtasks, streams (backpressure), buffers, cluster module & worker threads, CommonJS vs ESM
- **Why this exists:** Explains why Node handles 10k concurrent connections but chokes on CPU-heavy work
- **Industry relevance:** Debugging event-loop blocking, understanding when to use worker threads vs child processes
- **Misconceptions cleared:** "Node is multi-threaded because it's fast at concurrency" (it's single-threaded JS + a thread pool for I/O)
- **Practical Exercise:** Write a CPU-bound endpoint that blocks the event loop; fix it with `worker_threads`; measure before/after
- **Difficulty:** Hard
- **Prerequisites:** 1.3

Module 2.2 — Express Architecture (and Why Middleware Exists)

- **Submodules:** Request/response cycle, middleware as a chain-of-responsibility pattern, error-handling middleware, router composition, why middleware ordering matters
- **Why this exists:** Middleware is Express's core abstraction; misunderstanding it causes silent auth bypasses and error-handling bugs
- **Practical Exercise:** Build a custom logging + auth + rate-limit middleware stack from scratch (no libraries), then intentionally misorder it and diagnose the bug
- **Difficulty:** Medium
- **Prerequisites:** 2.1

Module 2.3 — API Design: REST Done Properly

- **Submodules:** Resource modeling, statelessness, HATEOAS (know it, know why most APIs skip it), versioning strategies, pagination (offset vs cursor), filtering/sorting conventions, idempotency keys for POST
- **Why this exists:** Most "REST APIs" in tutorials are RPC-over-HTTP; real REST design prevents entire categories of client-side bugs
- **Industry relevance:** API design mistakes are expensive to fix once clients depend on them
- **Practical Exercise:** Redesign a poorly modeled CRUD API (verbs-in-URLs, no pagination) into a proper resource-oriented design
- **Difficulty:** Medium
- **Prerequisites:** 1.1, 2.2

Module 2.4 — GraphQL (Comparative Study, Not Replacement)

- **Submodules:** Schema-first design, resolvers, N+1 problem & DataLoader, over-fetching/under-fetching problem REST has, when GraphQL is the wrong choice
- **Why this exists:** You need to justify architectural choices, not cargo-cult "GraphQL is better"
- **Practical Exercise:** Implement the same feature in REST and GraphQL; write a tradeoff memo (this is literally how staff engineers make ADRs)
- **Difficulty:** Medium
- **Prerequisites:** 2.3

Module 2.5 — Databases: Why They Exist, Then MongoDB, Then SQL for Contrast

- **Submodules:**
 - Why databases exist (durability, concurrency control, ACID) before touching any DB
 - MongoDB internals: BSON, WiredTiger storage engine basics, document model tradeoffs, indexing (B-tree), query planner, aggregation pipeline as a data-processing DSL
 - Schema design for NoSQL: embedding vs referencing, denormalization tradeoffs
 - SQL fundamentals for comparison: normalization (1NF–3NF), joins, transactions, when relational beats document model
 - Indexing theory: why an index speeds reads but slows writes, composite index ordering
- **Why this exists:** Picking MongoDB "because MERN" without understanding relational tradeoffs is how startups end up rewriting their data layer at scale
- **Misconceptions cleared:** "NoSQL is always more scalable" (it trades consistency guarantees, not magic scale); "adding an index is always good"
- **Practical Exercise:** Model the same domain (e.g., an order system) in both MongoDB and PostgreSQL; write a memo on which you'd pick and why, including a scenario where you'd switch

- **Difficulty:** Hard
- **Prerequisites:** 2.3

Module 2.6 — Authentication & Authorization

- **Submodules:** Why auth is hard (it's a distributed trust problem, not a login form), session-based auth vs JWT (tradeoffs, not "JWT is stateless so better"), token refresh strategies, OAuth2/OIDC flows explained end-to-end, RBAC vs ABAC, password hashing (bcrypt/argon2 — why salting matters), CSRF vs XSS as distinct threats
- **Why this exists:** Auth bugs are the most common source of real security incidents in junior-built systems
- **Industry relevance:** Every production system needs this; interview staple
- **Practical Exercise:** Implement JWT auth with refresh token rotation from scratch; then attack your own implementation (replay a stolen token, test for missing expiry checks)
- **Difficulty:** Hard
- **Prerequisites:** 2.5, Module 4.2 (Security, can be studied in parallel)

Skills after Phase 2: You can design, justify, and defend a backend architecture — not just make endpoints that return 200.

PHASE 3 — Frontend Engineering (React, Properly)

Module 3.1 — Why React Exists (Before Learning React)

- **Submodules:** Direct DOM manipulation problems at scale, declarative vs imperative UI, the reconciliation problem
- **Why this exists:** Students who learn React syntax first never understand *why* hooks rules exist
- **Prerequisites:** 1.2

Module 3.2 — React Internals

- **Submodules:** Virtual DOM & diffing algorithm, reconciliation, Fiber architecture (why it replaced the old stack reconciler — interruptible rendering), scheduling & priority lanes, render phase vs commit phase, why hooks have rules (linked list under the hood, not "just follow the rules")
- **Why this exists:** Explains stale closures in `useEffect`, why key props matter, why state updates batch
- **Misconceptions cleared:** "useEffect runs after every render" (dependency array semantics, cleanup timing); "React re-renders the whole DOM" (it diffs and patches)
- **Practical Exercise:** Predict re-render behavior for a component tree given specific state changes; fix a stale-closure bug in a given `useEffect`

- **Difficulty:** Hard
- **Prerequisites:** 3.1, 1.3

Module 3.3 — State Management: The Real Problem

- **Submodules:** Local vs lifted vs global state (decision framework, not library preference), why Redux's pattern (unidirectional flow) exists, Context API pitfalls (re-render cascades), server state vs client state as distinct problems (why TanStack Query exists — caching, dedup, invalidation, not just "fetching data")
- **Why this exists:** Most state-management bugs come from conflating server state with client state
- **Practical Exercise:** Refactor a prop-drilled component tree using the right tool for each type of state (not "just add Redux")
- **Difficulty:** Medium-Hard
- **Prerequisites:** 3.2

Module 3.4 — Performance Engineering in React

- **Submodules:** Memoization (`useMemo/useCallback/React.memo` — when they help vs add overhead), code splitting & lazy loading, virtualization for long lists, identifying unnecessary re-renders with React DevTools Profiler
- **Practical Exercise:** Profile a slow component tree; fix it with the minimum necessary memoization (over-memoizing is itself an anti-pattern to flag)
- **Difficulty:** Medium
- **Prerequisites:** 3.3

Module 3.5 — Next.js: Rendering Strategies as an Engineering Decision

- **Submodules:** CSR vs SSR vs SSG vs ISR (tradeoffs: TTFB, SEO, staleness, server cost), hydration and why it's expensive, App Router & Server Components (what actually runs where), streaming
- **Why this exists:** Rendering strategy is a business/performance tradeoff, not a framework default
- **Practical Exercise:** Given four different page types (blog, dashboard, product page, admin panel), justify a rendering strategy for each
- **Difficulty:** Hard
- **Prerequisites:** 3.4, 2.3

Skills after Phase 3: You choose rendering and state strategies based on tradeoffs you can articulate, not framework defaults.

PHASE 4 — Software Architecture & Craftsmanship

Module 4.1 — Clean Code, SOLID, and Design Patterns (Applied, Not Academic)

- **Submodules:** SOLID with real MERN examples (not [Animal/Dog](#)), common patterns actually used in Node/React (Factory, Strategy, Observer, Repository, Dependency Injection), when patterns are overkill
- **Why this exists:** Design patterns misapplied are worse than no patterns; the goal is recognizing the *problem shape*, not memorizing UML
- **Practical Exercise:** Refactor a God-object Express controller into a layered architecture (controller/service/repository) and justify each boundary
- **Difficulty:** Medium-Hard
- **Prerequisites:** 2.2, 3.3

Module 4.2 — Security Engineering

- **Submodules:** OWASP Top 10 explained by mechanism (not list memorization): injection, XSS, CSRF, SSRF, broken auth, insecure deserialization; rate limiting, input validation as a boundary discipline, secrets management
- **Practical Exercise:** Perform a mini security audit on a provided vulnerable Express app; patch each vulnerability with justification
- **Difficulty:** Hard
- **Prerequisites:** 2.6

Module 4.3 — Testing as an Engineering Discipline

- **Submodules:** Testing pyramid (unit/integration/e2e — real ratios, not dogma), why mocking exists and its dangers, TDD as a design tool (not a mandate), contract testing for APIs, flaky test root causes
- **Practical Exercise:** Write a test suite for an auth flow covering unit, integration, and one e2e test; intentionally introduce a flaky test and fix it
- **Difficulty:** Medium
- **Prerequisites:** 2.6, 3.3

Module 4.4 — System Design Fundamentals

- **Submodules:** Load balancing, caching layers (CDN, Redis, application cache), horizontal vs vertical scaling, database replication & sharding basics, message queues & eventual consistency, CAP theorem (practically, not just the triangle diagram), rate limiting algorithms
- **Why this exists:** This is the actual differentiator between a "coder" and a "software engineer" in interviews and in production
- **Practical Exercise:** Design a URL shortener and a ticket-booking system (classic interview problems) with explicit tradeoff discussion — directly relevant to your own booking-platform project

- **Difficulty:** Hard
- **Prerequisites:** 2.5, 4.2

Skills after Phase 4: You can defend architectural decisions in a design review, not just make code that runs.

PHASE 5 — DevOps & Production Engineering

Module 5.1 — Git at an Engineering Level

- **Submodules:** Git internals (commits as snapshots, not diffs; how branches are pointers), rebase vs merge (when each is correct), interactive rebase for clean history, resolving real conflicts, bisect for bug hunting
- **Practical Exercise:** Use `git bisect` to find a regression-introducing commit in a seeded repo
- **Difficulty:** Medium
- **Prerequisites:** 0.3

Module 5.2 — Docker & Containerization

- **Submodules:** Why containers exist (dependency/environment consistency problem), images vs containers, layers & caching, multi-stage builds, docker-compose for local dev parity with prod
- **Practical Exercise:** Containerize a full MERN app with multi-stage builds optimized for image size
- **Difficulty:** Medium
- **Prerequisites:** 0.3, 2.1

Module 5.3 — CI/CD Pipelines

- **Submodules:** Pipeline stages (lint/test/build/deploy), why CI exists (fast feedback loop), deployment strategies (blue-green, canary, rolling)
- **Practical Exercise:** Build a GitHub Actions pipeline that runs tests, builds a Docker image, and deploys to a staging environment on merge
- **Difficulty:** Medium
- **Prerequisites:** 5.2, 4.3

Module 5.4 — Deployment & Cloud Infrastructure

- **Submodules:** Reverse proxies (Nginx), environment configuration & secrets, DNS/SSL in production, serverless vs containers vs VMs tradeoffs, edge computing (Cloudflare Workers context — directly applicable to your Hono.js/D1 experience)

- **Practical Exercise:** Deploy the same app two ways (containerized on a VM vs serverless edge) and compare cold-start/cost/latency tradeoffs
- **Difficulty:** Hard
- **Prerequisites:** 5.3

Module 5.5 — Observability: Monitoring, Logging, Tracing

- **Submodules:** Structured logging vs print debugging, metrics (RED/USE methods), distributed tracing (why it exists once you have >1 service — directly relevant to your saga-pattern booking platform), alerting without alert fatigue
- **Practical Exercise:** Instrument a multi-service app with OpenTelemetry tracing and diagnose a latency issue using trace spans
- **Difficulty:** Hard
- **Prerequisites:** 5.4

Skills after Phase 5: You can ship and operate software in production, not just make it work on localhost.

FINAL CAPSTONE PATH

A single capstone system, built incrementally, that forces integration of every phase:

"Distributed Multi-Tenant Marketplace/Booking Platform" (aligns with your existing project)

1. Design phase: system design doc with explicit tradeoffs (Phase 4)
 2. Data layer: MongoDB + Postgres hybrid, justified (Phase 2.5)
 3. Backend: REST + GraphQL comparison for one module (Phase 2)
 4. Auth: full JWT + RBAC (Phase 2.6, 4.2)
 5. Frontend: Next.js with mixed rendering strategies (Phase 3.5)
 6. Distributed systems: saga pattern, eventual consistency, circuit breakers (Phase 4.4)
 7. Testing: full pyramid (Phase 4.3)
 8. Containerized, CI/CD'd, deployed with observability (Phase 5)
 9. Load-tested and profiled, with a written postmortem of one deliberately induced production incident
-

RECOMMENDED LEARNING SEQUENCE

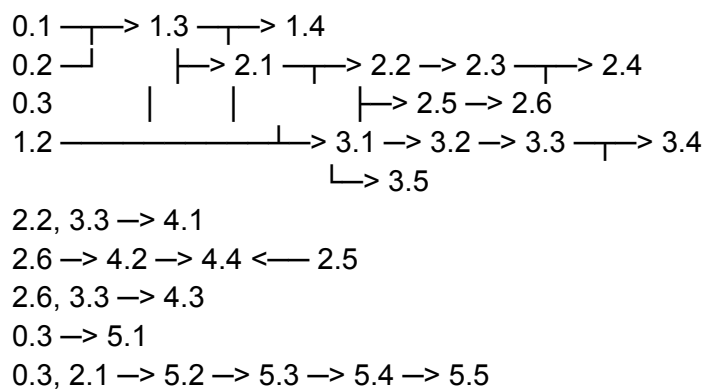
Phase 0 → Phase 1 → Phase 2 → Phase 3 → Phase 4 → Phase 5 → Capstone

Within Phase 2 and 3, submodules can interleave once their direct prerequisite is met (e.g., 2.4 GraphQL can be studied alongside 3.3 State Management since both depend only on 2.3/3.2 respectively).

SKILLS ACQUIRED AFTER EACH PHASE

Phase	Core Skill Shift
0	Stops treating runtime/network as magic
1	Debugs at language/browser level, not by guessing
2	Designs and defends backend architecture
3	Chooses rendering/state strategy by tradeoff, not default
4	Survives a design review; writes maintainable systems
5	Ships and operates software in production

PREREQUISITE GRAPH (condensed)



SUGGESTED MILESTONES

1. Explain the JS event loop and predict async output correctly, unaided
2. Build and secure a JWT-authenticated REST API with proper resource modeling

3. Diagnose a React re-render performance issue using the Profiler
 4. Design a system (URL shortener, booking system) on a whiteboard with tradeoffs stated aloud
 5. Containerize, CI/CD, and deploy a service with tracing and alerting
 6. Ship the capstone and survive a self-induced incident postmortem
-

COMMON BEGINNER MISTAKES PREVENTED

- Treating `===` vs `==` as the whole coercion story
 - Believing React "re-renders the DOM" instead of diffing/patching
 - Using JWT because "it's stateless" without understanding revocation tradeoffs
 - Choosing MongoDB by default without modeling the relational alternative
 - Adding Redux/global state before establishing it's actually server state
 - Writing tests that mock everything and verify nothing real
 - Deploying without observability, then debugging prod via `console.log` and hope
-

PRODUCTION ENGINEERING SKILLS GAINED

- Root-causing incidents using traces/logs/metrics instead of guesswork
- Writing architecture decision records (ADRs) with explicit tradeoffs
- Designing systems that degrade gracefully under partial failure
- Reviewing code for security and maintainability, not just "does it run"
- Making build-vs-buy and scale-now-vs-later calls defensibly